



Ansible

Ansible is an open-source automation tool used for configuration management, application deployment, orchestration, and task automation. It allows users to automate the management of systems, networks, and applications in a simple and effective way. Ansible uses a declarative language to describe the desired state of systems, which means users can specify what they want their systems to look like rather than how to achieve that state.

Need of Ansible

Simplified Automation: Ansible makes automating IT tasks straightforward and manageable.

Agentless Management: It uses SSH for Unix-like systems and WinRM for Windows, eliminating the need for agent installations.

Idempotence: Ensures that running the same playbook multiple times yields the same state, reducing the chances of configuration drift.

Scalability: Can manage hundreds or thousands of machines simultaneously, making it suitable for large-scale environments.

Components of Ansible

Control Node: The machine where Ansible is installed and executed.

Managed Nodes: Servers or devices that Ansible controls.

Inventory: A list of managed nodes and their connection details.

Playbooks: YAML files that define automation tasks and configurations.

Modules: Reusable scripts that perform specific tasks on managed nodes.

YAML

YAML (YAML Ain't Markup Language) is a human-readable data serialization format commonly used for configuration files. Its syntax is simple and focuses on data structures, making it easy to read and write. It uses python and indentation-sensitive.

Inventory

Inventory is a file or script that defines the hosts managed by Ansible. It can be written in several ways:

Static Inventory: A text file listing hosts (usually /etc/ansible/hosts).

Dynamic Inventory: Generated dynamically using scripts or plugins.

Example of static inventory

```
[webservers]
192.168.1.10
192.168.1.11
```

```
[databases]
db1 ansible_host=192.168.1.15
```

Ansible Configuration

The Ansible configuration file (ansible.cfg) customizes various aspects of Ansible's operation, including default inventory locations, SSH parameters, and logging options.

Example of an ansible.cfg:

```
[defaults]
inventory = /etc/ansible/hosts
remote_user = ansible
```

YAML File structure

YAML structure is based on indentation and uses key-value pairs to represent data. Each entry is defined by a colon and space.

Example

```
- name: Install a package
  hosts: webservers
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
```

hosts:all- means it will get installed on all server

Variables

Variables can be defined in playbooks, inventory files, or separate variable files.

1) In a playbook

```
---
- hosts: localhost
  vars:
    my_variable: "Hello, Ansible!"
  tasks:
    - name: Print my_variable
      debug:
        msg: "{{ my_variable }}"
```

2) In an inventory file

```
[my_group]
server1 ansible_host=192.168.1.1 my_variable="Hello from Inventory"
```

3) In a Separate Variable File (e.g., vars.yml)

```
my_variable: "Hello from variable file"
```

After that include this file in a playbook

```
- hosts: localhost
  vars_files:
    - vars.yml
  tasks:
    - debug:
        msg: "{{ my_variable }}"
```

Arrays(Lists)

Lists (arrays) in Ansible are defined using a dash (-) followed by the item.

```
---
- hosts: localhost
  vars:
    my_list:
      - item1
      - item2
      - item3
```

Dictionaries (Maps) in Ansible

Dictionaries (or maps) are key-value pairs in Ansible.

```
---
- hosts: localhost
  vars:
    my_dict:
      key1: value1
      key2: value2
      key3: value3
```

Loops

Loops can be utilized to iterate through both lists and dictionaries, making your playbooks more

dynamic and reusable. Ansible supports looping through items in lists or dictionaries. You can use the loop keyword or with_items.

Using loops with lists

```
---
- hosts: localhost
  vars:
    my_list:
      - apple
      - banana
      - cherry
  tasks:
    - name: Print fruit
      debug:
        msg: "I like {{ item }}"
      loop: "{{ my_list }}"
```

Using loops with dictionaries

```
---
- hosts: localhost
  vars:
    my_dict:
      name: Alice
      age: 30
      city: Wonderland
  tasks:
    - name: Print each key-value pair
      debug:
        msg: "{{ item.key }}: {{ item.value }}"
      loop: "{{ my_dict | dict2items }}"
```

To Ping Local Host Code

To test connectivity to the local host, use the ping module.

```
ansible localhost -m ping
```

Or

```
- name: Ping all hosts
  hosts: localhost
  tasks:
    - name: Ping the local machine
      ping:
```

Overview of Ansible Playbook

An Ansible playbook consists of:

name: A description of what the playbook does.

hosts: Specifies the target hosts.

tasks: A list of actions to perform.

```
- name: Install Apache on web servers
  hosts: webservers
  tasks:
    - name: Install Apache
      yum:
        name: httpd
        state: present
      become: yes # Run with sudo privileges
```

Ansible Modules

Ansible modules are reusable scripts that perform specific tasks on managed nodes. Common module categories include:

System Modules: Manage system services (e.g., service, user).

File Modules: Manipulate files (e.g., copy, file, template).

Database Modules: Interact with databases (e.g., mysql_db, postgresql_db).

Cloud Modules: Manage cloud resources (e.g., ec2 for AWS, azurerm for Azure).

Write a Playbook for Installing and Starting a Package

For installing in redhat use yum and for ubuntu use apt

```
- name: Install and start Apache
  hosts: webservers
  tasks:
    - name: Install httpd
      yum:
        name: httpd
        state: present
    - name: Start httpd service
      service:
        name: httpd
        state: started
```

This playbook installs the Apache HTTP server on specified hosts and ensures the service is started.

Write a Playbook for Copying Files on Remote

Server

For copying `copy` is used.

```
- name: Copy files to remote server
hosts: remote_servers
tasks:
  - name: Copy index.html
    copy:
      src: ./index.html
      dest: /var/www/html/index.html
```

This playbook copies a local HTML file to the specified path on the remote server.

Code for Taking Backup of Files

Using backup: yes within the copy module to keep a backup of existing files before overwriting.

```
- name: Backup and copy file
hosts: webserver
tasks:
  - name: Copy file with backup
    copy:
      src: /path/to/local/file
      dest: /path/to/remote/file
      backup: yes
```

Ansible File Modules

Ansible's file module helps manage files and directories.

```
- name: Manage files and directories
hosts: localhost
tasks:
  - name: Create a directory
    file:
      path: /tmp/example_dir
      state: directory
      mode: '0755'

  - name: Create a file
    file:
      path: /tmp/example_dir/example_file.txt
      state: touch
      mode: '0644'

  - name: Delete a file
    file:
      path: /tmp/example_dir/example_file.txt
      state: absent
```

This example demonstrates creating a directory, creating a file within it, and deleting the file.

mode: Sets the permissions for the file on the remote machine. 0644 means:

6 (read and write) for the owner.

4 (read-only) for the group.

4 (read-only) for others.

Ansible Plugins

Plugins in Ansible extend functionality and are categorized into action, callback, connection, lookup, filter, inventory, and vars plugins.

Action Plugins: Modify tasks (e.g., `command`).

Callback Plugins: Customize output, such as `logs`.

Connection Plugins: Control connection protocols (e.g., `ssh`).

Lookup Plugins: Fetch external data (e.g., `from files`).

Filter Plugins: Manipulate variables (e.g., `to_json`).

Inventory Plugins: Manage inventory sources dynamically (e.g., `AWS`, `Azure`).

Vars Plugins: Manage variable sources.

Example of Lookup Plugin:

```
- name: Fetch data from a file
debug:
  msg: "{{ lookup('file', '/path/to/file.txt') }}"
```

`lookup('file')` reads content from the specified file, allowing dynamic data integration into playbooks

Creating and disabling Cron job

Using Ansible's cron module, you can easily automate the creation and management of cron jobs on your servers. This allows you to ensure that important tasks are executed regularly without manual intervention, enhancing the efficiency and reliability of your automation processes.

Create a cron job that runs a script daily at 2 AM

```
---
- hosts: localhost
  tasks:
    - name: Create a cron job to run a script daily at 2 AM
      cron:
        name: "Run my script"
        minute: "0"
        hour: "2"
        job: "/path/to/my/script.sh"
        state: present # Indicates the cron job should be created
```

Disabling a Cron Job

```
---
- hosts: localhost
  tasks:
    - name: Disable the cron job
      cron:
        name: "Run my script"
        state: absent # Indicates the cron job should be removed
```

Explanation of Parameters

name: A unique name for the cron job, used to identify it in the cron table.

minute: The minute when the job should run (0-59).

hour: The hour when the job should run (0-23).

job: The command to execute. This can be a shell command or a path to a script.

state:

present indicates that the cron job should be created.

absent indicates that the cron job should be removed.

In Ansible, specifying **state: absent** indicates that the resource (like a user or package) should be removed.

User Management (Create, Delete, Password Setup)

Creating a User on a Remote System

```
- name: Create a user
  user:
    name: "john"
    state: present
    groups: "sudo"
    shell: "/bin/bash"
```

Deleting a User

```
- name: Delete a user
  user:
    name: "john"
    state: absent
```

Setting a Password for a User

Ansible can manage user passwords by encrypting them.

```
- name: Add user with a password
  user:
    name: "john"
    password: "{{ 'password123' | password_hash('sha512') }}"
```

state: present/absent manages whether the user is created or deleted.

password_hash secures the password.

Process Management

Find and Stop a Process: Use the **shell** module to find and stop a process. For example, stopping a process by name.

```
- name: Find and stop a process
  shell: "pkill -f process_name"
  args:
    warn: false
```

pkill -f is used to find processes by name and stop them. **warn: false** suppresses warnings about shell commands.

Downloading Files from the Internet

Use the `get_url` module to download files from the internet.

```
- name: Download a file
  get_url:
    url: "http://example.com/file.tar.gz"
    dest: "/tmp/file.tar.gz"
```

`get_url` is ideal for downloading resources from external URLs.

`dest` specifies the location where the file should be saved.

Enabling Firewall Service

```
- name: Enable firewall service
  service:
    name: firewalld
    state: started
    enabled: yes

- name: Allow HTTP service through the firewall
  firewalld:
    service: http
    permanent: true
    state: enabled
    immediate: true
```

`firewalld` is used to manage firewall rules on Linux systems.

`immediate: true` applies the rule immediately without requiring a restart.

Executing Basic Ansible Commands within Playbooks

```
- name: Update and upgrade system packages
  apt:
    update_cache: yes
    upgrade: dist
```

`update_cache` updates the package index.

`upgrade: dist` upgrades all packages to their latest versions.

Service Management

Ansible can manage services by starting, stopping, restarting, or updating them

Starting and Enabling a Service

```
- name: Start and enable the NGINX service
  service:
    name: nginx
    state: started
    enabled: yes
```

Restarting or Reloading a Service

```
- name: Restart the NGINX service
  service:
    name: nginx
    state: restarted
```

The `service` module is used for service management.

`enabled: yes` ensures the service starts at boot.

Ansible Ad-Hoc Tasks

Ad-hoc tasks are one-liners executed without a playbook, usually for quick actions. They are run from the command line

Example of an Ad-Hoc Command

```
ansible all -m ping
```

`-m ping` uses the ping module to test connectivity to all hosts in the inventory

Example

```
ansible web -m apt -a "name=nginx state=present"
```

This installs NGINX on all hosts in the web group.

Tags in Ansible

Tags allow selective task execution within a playbook. This is useful when you only want to run specific parts of a playbook.

```
- name: Install NGINX
  apt:
    name: nginx
    state: present
  tags:
    - webservers

- name: Start NGINX service
  service:
    name: nginx
    state: started
  tags:
    - webservers
    - service
```

Running tasks by tag

```
ansible-playbook playbook.yml --tags "webservers"
```

Tags allow tasks with specific tags to be run independently.

This approach is efficient in long playbooks where only a few steps need to be executed.

Conditions in Ansible

Conditional Execution allows tasks to run based on certain conditions, using the when clause.

```
- name: Install NGINX on Debian systems
  apt:
    name: nginx
    state: present
  when: ansible_os_family == "Debian"
```

The when statement checks if the OS family is Debian, and if true, installs NGINX. This enables platform-specific configurations.

Built-in Variables provide system and environment information. Examples include `ansible_os_family`, `ansible_hostname`, `ansible_processor`, etc.

Jinja2 Templating

Jinja2 is a powerful templating engine used in Ansible to dynamically create content in configuration files, scripts, or even task parameters. Jinja2 allows you to use variables, loops, conditionals, filters, and more in templates, which

makes configurations more flexible and adaptable to different environments.

Basic Syntax

Variables: `{{ variable_name }}` - Enclosed within double curly braces `{{ }}`

Control Structures: `{% ... %}` for loops and conditions.

Common Filters

upper: `{{ variable | upper }}` - Converts to uppercase.

default: `{{ variable | default("default_value") }}` - Uses a default if undefined.

length: `{{ list_variable | length }}` - Gets the length.

join: `{{ list_variable | join(", ") }}` - Joins list items with a separator.

Conditionals

```
{% if condition %}
    # Code if true
{% else %}
    # Code if false
{% endif %}
```

Loops

```
{% for item in items %}
    {{ item }}
{% endfor %}
```

Use `loop.index`, `loop.length`, and `loop.last` for loop control.

Handlers

Handlers are used to trigger actions only when a change occurs. A handler will only execute if a task notifies it.

```
- name: Restart Apache
  service:
    name: apache2
    state: restarted
  notify: restart apache

handlers:
- name: restart apache
  service:
    name: apache2
    state: restarted
```

Ansible Roles

Roles allow modular organization of playbooks, making them reusable and easier to maintain. Roles are structured directories containing tasks, handlers, variables, files, templates, and default configurations.

Creating an Ansible Role

```
ansible-galaxy init role_name
```

Problem Statement: Suppose we need a role to configure an Apache web server.

Role Structure

```
role_name/
├─ tasks/main.yml
├─ handlers/main.yml
├─ templates
├─ files
├─ vars/main.yml
└─ defaults/main.yml
```

Playbook Using the Role

```
- hosts: web_servers
  roles:
    - role_name
```

Roles enhance reusability. Here, **role_name** can be applied to any host group needing Apache configuration.

Ansible Galaxy

Ansible Galaxy is a community hub for finding, sharing, and managing Ansible roles. It enables downloading community roles for common tasks like configuring web servers, databases, or cloud resources.

Installing a Role from Galaxy

```
ansible-galaxy install username.role_name
```

Galaxy helps extend Ansible's capabilities by allowing access to reusable roles.

Ansible Vault

Ansible Vault is a feature to secure sensitive data like passwords or keys within playbooks, ensuring they are encrypted and secure.

Creating an Encrypted File

```
ansible-vault create secrets.yml
```

Editing an Encrypted File

```
ansible-vault edit secrets.yml
```

Using Vault in a Playbook

```
- hosts: all
  vars_files:
    - secrets.yml # Loads encrypted variables
  tasks:
    - name: Print sensitive data
      debug:
        msg: "{{ secret_password }}"
```

Ansible Strategies

Strategies define how Ansible executes tasks.

Linear Strategy:

This is the default strategy in Ansible where tasks are executed sequentially in the order they appear in the playbook.

Free Strategy:

This strategy executes tasks on all hosts at the same time, without any coordination or order.

Serial Strategy:

This strategy executes tasks on a defined number of hosts at a time, allowing for controlled parallelism.

Batch Strategy:

This strategy executes tasks in batches, based on a defined percentage or number of hosts.

Rolling Strategy:

This strategy executes tasks on a set of hosts in a rolling fashion, updating one host at a time while keeping the service operational.

Example-free strategy

```
- hosts: all
  strategy: free
  tasks:
    - name: Example task
      command: /bin/echo "Running tasks freely"
```

Ansible Collections

Collections are bundles of modules, plugins, roles, and playbooks that extend Ansible's functionality. They allow sharing and reuse of automation content.

Installing a Collection

```
ansible-galaxy collection install community.mysql
```

Using a Module from a Collection

```
- hosts: database
  tasks:
    - name: Create MySQL database
      community.mysql.mysql_db:
        name: mydb
        state: present
```

Error Handling in Ansible

Error handling in Ansible is a critical feature that allows you to manage and respond to errors that occur during playbook execution. This helps ensure that your automation processes are robust, resilient, and capable of handling unexpected situations.

Ignoring Errors

You can specify that a task should ignore errors and continue executing subsequent tasks. This is done using the `ignore_errors` directive.

```
- name: This task may fail
  command: /bin/false
  ignore_errors: yes
```

In this example, even if the command fails the playbook will continue to execute the next tasks.

Error Handling Blocks

Ansible provides the ability to group tasks into blocks. If a task in a block fails, you can specify actions to take using the `rescue` section to handle the failure and an `always` section to perform tasks that should run regardless of success or failure.

```
- block:
  - name: Attempt to perform an action
    command: /bin/false
  rescue:
    - name: Handle the failure
      debug:
        msg: "The previous task failed, but we are handling it gracefully."
  always:
    - name: This will always run
      debug:
        msg: "Cleanup actions or other necessary steps."
```

In this example, if the command in the block fails, the tasks in the `rescue block` will execute, followed by the tasks in the `always` block.

Conditional Execution

You can also handle errors conditionally by using the `when` clause based on the result of previous tasks.

```
- name: Attempt to create a user
  user:
    name: newuser
    state: present
  register: user_creation

- name: Handle user creation failure
  debug:
    msg: "User creation failed"
  when: user_creation.failed
```

Retry Mechanism

Ansible allows you to retry tasks that might fail due to transient issues. You can use the retries and delay parameters to specify how many times to retry a task and how long to wait between retries

```
- name: Ensure a service is running
  service:
    name: my_service
    state: started
  register: service_status
  until: service_status is succeeded
  retries: 5
  delay: 10
```

In this case, the task will attempt to start the service up to 5 times, waiting 10 seconds between each attempt.

Dynamic Inventory

Dynamic inventory is used for infrastructure that changes frequently, such as cloud servers. It can be generated by scripts that query cloud APIs.

Asynchronous Actions in Ansible

Asynchronous actions in Ansible allow you to run long-running tasks without blocking the playbook's execution. This is useful for operations that may take time, like starting a service or running a script. By using the `async` parameter, you can specify how long to wait for the task to finish, while the `poll` parameter determines how frequently Ansible checks the status of the task.

async: Maximum time (in seconds) that Ansible should wait for the task to complete.

poll: Frequency of checking the task status; set to 0 for non-blocking execution.

```
- hosts: all
  tasks:
    - name: Start a long-running service
      command: /path/to/start-service.sh
      async: 600 # Allow up to 10 minutes for the task
      poll: 0   # Do not wait for the task to complete

    - name: Wait for a bit before checking status
      pause:
        seconds: 30 # Pause for 30 seconds

    - name: Check if the service is running
      command: /path/to/check-service-status.sh
      register: service_status

    - name: Output service status
      debug:
        msg: "Service is running: {{ service_status.stdout }}"
```

Explanation of the Code

1. **Start the Service:** The first task starts a long-running service using a shell script. It runs asynchronously for a maximum of 10 minutes (`600 seconds`), and does not block execution of subsequent tasks (`poll: 0`).
2. **Pause:** The playbook pauses for 30 seconds to give the service time to start.
3. **Check Status:** After the pause, it checks the service status using another command.
4. **Output Status:** Finally, it prints the status of the service using the `debug` module.

Creating a Virtual Machine on Azure

```
- hosts: localhost
  tasks:
    - name: Create VM in Azure
      azure_rm_virtualmachine:
        resource_group: myResourceGroup
        name: myVM
        vm_size: Standard_DS1_v2
        admin_username: azureuser
        admin_password: Password123!
        image:
          offer: UbuntuServer
          publisher: Canonical
          sku: 18.04-LTS
          version: latest
```

Ansible Tower

Ansible Tower is an enterprise framework for controlling, securing, and managing Ansible automation with a web-based interface.

Features it include-

RBAC (Role-Based Access Control): Controls user access.

Workflow Automation: Orchestrates multi-step processes.

Logging: Provides detailed activity logs.

Inventory Management: Centralizes and manages inventory dynamically.

Ansible Tower is especially useful in enterprise environments to manage complex automation workflows, ensure security, and track changes.

Creating an Instance on AWS

```
- hosts: localhost
  tasks:
    - name: Launch an EC2 instance
      amazon.aws.ec2_instance:
        key_name: my_key
        instance_type: t2.micro
        image_id: ami-0abcdef1234567890
        wait: yes
        region: us-west-2
```

Echo

```
---
- name: Echo command playbook
  hosts: all                # Target all servers in the inventory
  tasks:
    - name: Run echo command
      command: echo "Hello Anupam" # The command to run
      register: echo_output # Register the output of the command

    - name: Display the output
      debug:
        msg: "{{ echo_output.stdout }}" # Display the output from the command
```

Ansible Adhocs

Ansible ad hoc commands enable you to perform tasks on remote nodes without having to write a playbook. They are very useful when you simply need to do one or two things quickly and often, to many remote nodes.

Syntax of Ansible Ad-hoc Commands

```
ansible [pattern] -m [module] -a "[arguments]"
```

- **[pattern]:** This specifies the target hosts, which can be an individual host, a group of hosts, or all hosts defined in the inventory.
- **-m [module]:** This specifies the Ansible module to use (e.g., ping, shell, copy, etc.).
- **-a "[arguments]":** This includes the arguments to pass to the module being used.

Command	Description
<code>ansible all -m ping</code>	Pings all hosts in the inventory to check connectivity.
<code>ansible webservers -m shell -a "uptime"</code>	Executes the <code>uptime</code> command on all web servers.
<code>ansible dbbservers -m copy -a "src=/tmp/file dest=/var/tmp/"</code>	Copies a file from the control machine to all database servers.
<code>ansible all -m yum -a "name=httpd state=present"</code>	Installs the <code>httpd</code> package on all hosts.
<code>ansible all -m service -a "name=httpd state=started"</code>	Starts the <code>httpd</code> service on all hosts.
<code>ansible all -m user -a "name=john state=present"</code>	Creates a user named <code>john</code> on all hosts.
<code>ansible all -m apt -a "name=git state=absent"</code>	Uninstalls the <code>git</code> package from all hosts.
<code>ansible all -m fetch -a "src=/etc/passwd dest=/tmp/passwd"</code>	Fetches the <code>/etc/passwd</code> file from all hosts to the control machine.
<code>ansible all -m command -a "ls /tmp"</code>	Executes the <code>ls</code> command in the <code>/tmp</code> directory on all hosts.
<code>ansible all -m debug -a "var=ansible_hostname"</code>	Displays the hostname of all managed nodes.